

Manual Testing — How to Write Findings

Report like a user, not like a developer. A universal guide for QA testers.

Why this matters: Manual testing is done from the *user's* side of the screen. A tester's job is to describe what a real user would see, feel, and struggle with — not to diagnose the code. Technical-sounding reports confuse the team and miss the actual user pain. This guide shows how to write findings in clear, user-perspective language.

1. Core Principles

- **Describe what you see, not what you assume is broken underneath.**

Don't guess at causes. Just describe the observable behaviour.

- **Write so a non-technical person can understand.**

Imagine explaining to a family member who has never used the app. If they can picture it, you wrote it right.

- **Use real-world, app-surface words.**

Say 'button', 'screen', 'message', 'popup', 'text', 'image', 'loading circle' — not internal names, file names, or code terms.

- **Report the impact on the user, not the cause in the code.**

'User cannot continue' is more useful than any technical explanation of why.

- **One bug, one report. Keep it small and specific.**

Don't bundle five issues into one note. Each observation gets its own entry.

- **Always include steps, expected, and actual.**

Without these three, the developer cannot reproduce or fix.

- **Attach a screenshot or screen recording for every finding.**

A picture removes 90% of back-and-forth.

2. Technical vs User-Perspective (Rephrasing Examples)

The left column is how a developer or code-reviewer might write a finding. The right column is how the **same issue** should be written by a manual tester. Always write like the right column.

■ Technical (avoid)	■ User-perspective (write like this)
Null pointer exception on list adapter when data empty.	App crashes when the list has no items. A blank screen appears for a second, then the app closes.
API returns 500 on submit; no error handler in UI.	When I tap Submit, nothing happens. No success message, no error message. I don't know if it worked or not.
onPause / onResume not implemented — background streaming continues.	When I minimize the app and come back after 5 minutes, the timer is still running and my battery dropped a lot.
Token logged to console in debug build.	(Flag to developer privately — not a user-visible issue, but note it as a security concern.)
Hardcoded strings in code, not in resource file.	When I switch the app language to Hindi, some words on the screen are still in English.

■ Technical (avoid)	■ User-perspective (write like this)
Missing contentDescription on ImageView.	The screen reader does not read out what this icon means, so a visually impaired user cannot tell what it does.
No values-night resource directory.	When my phone is set to dark mode, the app still shows the bright/light theme. It doesn't follow my phone setting.
notifyDataSetChanged on every bind causes jank.	The list feels laggy when scrolling fast. Items seem to flicker or redraw.
maxLines=1 with ellipsize truncates long names.	Long names get cut off with '...' at the end. I cannot see the full name anywhere.
No retry mechanism on network failure.	When my internet is slow or off, the screen just keeps loading forever. There is no 'Try Again' option.
Agora UID mismatch — host leave not detected.	When the host leaves, other users stay stuck inside. Nothing tells them the session ended, and they can't exit normally.
Room details fetched once on entry, no polling.	If someone joins or leaves after I am already on the screen, I don't see the change until I exit and come back.

3. The Four Perspectives to Test From

Every feature should be reviewed through these four lenses. Findings can be tagged with the perspective they belong to.

A. UI Perspective — "What do I see?"

Focus on the visible surface of the screen. Ignore what happens underneath.

- Is everything aligned, spaced, and sized properly?
- Are colors, fonts, and icons consistent with the rest of the app?
- Is any text cut off, overlapping, or too small to read?
- Do images and icons load fully, or do I see broken placeholders?
- Does the screen look the same on small and large phones?
- Does it look correct in both portrait and landscape?
- Does dark mode (if supported) look right, or is something unreadable?

B. UX Perspective — "How does it feel to use?"

Focus on the journey, effort, and emotional reaction. A feature can work perfectly and still feel bad.

- Is it obvious what to do next, or do I have to guess?
- Are there too many taps to finish a simple action?
- Do error messages tell me what went wrong AND how to fix it?
- If something is loading, do I see a clear loading indicator?
- When an action succeeds, do I get clear feedback (tick, message, animation)?
- If I make a mistake, can I undo or go back easily?
- Does the screen feel rushed, cluttered, empty, or confusing?

C. Functional Perspective — "Does it actually do what it claims?"

Focus on behaviour. Every button, field, and action must deliver the promised outcome.

- Does every button do what its label says?
- Do all links, tabs, and menu items lead to the correct place?
- Do forms accept valid input and reject invalid input correctly?
- Do required fields block submission when empty?
- Do filters, sorts, and searches give the right results?
- Does data save and reload correctly after I close and reopen?
- Do shared/deep links open the correct screen?

D. Real-User Perspective — "What happens in real life?"

Focus on situations a real user will run into — not a perfect testing environment.

- Slow internet — does the screen recover, or does it freeze?
- No internet — is there a clear message and retry option?
- Incoming call or notification mid-action — does the app come back cleanly?
- Minimize the app for a few minutes, then open it — is my state preserved?
- Rotate the phone — does the layout adjust without breaking?
- Low battery / device sleep during a task — is data saved?
- Old phone with small screen or old OS version — is the app usable?
- Language set to Hindi / Tamil / other — is everything translated and fitting?

4. How to Write a Bug Report (User-Perspective Template)

Every finding should fill in these fields. Keep the language simple and describe what a normal user would see or feel.

Field	What goes here	Example
Title	One-line summary in plain words. What is wrong, where?	Submit button does nothing when tapped on the sign-up screen
Screen / Feature	Which screen or flow the issue is on.	Sign-up → Step 2 (Details screen)
Perspective	UI / UX / Functional / Real-User — pick one or two.	Functional + UX
Steps to Reproduce	Numbered, short steps any teammate can follow.	1. Open app 2. Tap 'Sign up' 3. Fill all fields 4. Tap Submit
Expected Result	What should happen, in user words.	Account should be created and the next screen should open.
Actual Result	What actually happens, in user words. No technical guessing.	Nothing happens. The button looks tapped but the screen doesn't change. No message appears.
Device	Brand + model.	Samsung Galaxy M14
OS Version	Android / iOS version.	Android 13
Build / App Version	Version being tested.	v2.4.1 (build 318)
Network	WiFi / 4G / Slow / Offline.	WiFi, good signal
Frequency	Always / Sometimes / Once.	Always (tried 5 times)
Severity	Blocker / Critical / Major / Minor / Cosmetic.	Blocker
Screenshot / Recording	Attach image or short screen recording.	sign_up_submit_fail.mp4
Extra Notes	Anything unusual: what you were doing, what happened just before.	Happens only after the phone was locked briefly during the form.

5. Severity — in User Language

Level	What it means (user view)
Blocker	User cannot use the feature at all. App crashes or a core action does nothing. Must be fixed before release.
Critical	Major feature is broken or data is lost. User can continue elsewhere but this part is unusable.
Major	Important issue, but a workaround exists. Users will complain but can still finish their task.
Minor	Small issue that does not stop the user. Mildly annoying, rarely noticed.
Cosmetic	Visual only. Alignment, color, spacing, typo. Doesn't affect functionality.

6. Golden Rules — Quick Reminders

- ✓ If a normal user wouldn't say it, don't write it.
- ✓ No code file names, no internal variable names, no framework terms.
- ✓ Describe the *experience*, not the *implementation*.
- ✓ One screenshot is worth ten paragraphs.
- ✓ If the developer has to ask a clarifying question, the report was incomplete.
- ✓ Report the symptom in the UI, not the suspected cause in the code.
- ✓ When in doubt, write it the way you would describe it to a friend over chat.

Use this document as the standard for all manual testing reports. When every tester writes findings from the user's perspective, the team spends less time translating and more time fixing real problems.